EPL EDGE LAB

AI + 통계 기반 EPL 확률 분석 플랫폼

Project Foundation / MVP Architecture / Development Kickoff Prompt

문서 목적	프로젝트 기획과 개발 착수를 위한 최소 공통 기준
현재 단계	아이디어 검증 -> MVP 설계
서비스 포지션	스포츠 베팅 사이트가 아닌 "축구 데이터/확률 분석 플랫폼"
작성 기준일	2026-10-15

핵심 원칙: AI 가 승패를 "찍는" 서비스가 아니라, 데이터 모델이 확률을 계산하고 AI 가 최신 비정형 정보를 구조화/설명한다.

1. 프로젝트 개요

EPL Edge Lab(가칭)은 잉글랜드 프리미어리그 경기의 일정, 팀 전력, 최근 폼, xG, 부상/징계, 예상 라인업, 배당 변화, 뉴스/인터뷰 등을 통합해 경기별 확률과 시장 대비 차이(edge)를 보여주는 웹 기반 분석 서비스다.

제품 정의 "정답 픽"을 판매하는 사이트가 아니라, 모델 확률과 근거를 투명하게 보여주는 스포츠 분석 도구로 포지셔닝한다.

1.1 목적

- 사용자가 하루 또는 주간 EPL 경기에서 2~3 개 경기를 선택하고, 각 경기의 승/무/패 확률과 배당 대비 기대값을 확인한다.
- 통계 모델을 핵심 엔진으로 두고, AI 는 뉴스/부상/감독 인터뷰 등 비정형 정보를 구조화하는 보조 계층으로 사용한다.
- 예측 이력과 모델 버전을 변경 불가능하게 축적해 장기적으로 "데이터 자산"과 신뢰도를 만든다.
- 초기에는 개인용/공개 베타로 검증하고, 성능과 트래픽이 확인되면 광고, 프리미엄 구독, API/B2B 리포트로 확장한다.

1.2 성공 기준

구분	초기 기준
제품	사용자가 경기 2 개를 선택해 1X2 확률, 시장 공정확률, Edge, EV, 조합 확률을 10 초 이내 확인
모델	Brier score / Log loss / Calibration 을 기준으로 baseline 대비 개선 확인
데이터	과거 예측값, 당시 배당, 모델 버전, 결과를 모두 시점 고정 저장
운영	데이터 수집 실패/지연이 UI 에 명확히 표시되고 재실행 가능
상용화	검색 유입이 가능한 경기/팀 페이지와 추적 가능한 분석 이벤트 구축

2. 제품 방향성과 원칙

2.1 핵심 사용자 흐름

1. 오늘/이번 주 EPL 경기 목록을 본다.
2. 사용자가 2~3 개 경기를 선택한다.
3. 서비스가 통계 모델 확률, 시장 공정확률(de-vig), Edge, EV, 신뢰도와 주요 변수를 표시한다.
4. 2 경기 조합의 경우 서로 다른 경기 기준으로 독립 근사 확률과 조합 배당/EV 를 계산한다.
5. 경기 종료 후 결과를 자동 반영하고 예측 성능 통계에 누적한다.

2.2 반드시 지킬 원칙

- 확률과 기대값을 제공하되 "확정", "무조건", "필승" 같은 표현을 사용하지 않는다.
- LLM 의 자연어 판단을 최종 승률로 직접 사용하지 않는다.
- 경기 시작 전 생성된 예측은 이후 재계산 결과로 덮어쓰지 않고 버전별로 보존한다.
- 배당은 북메이커 마진을 제거한 시장 공정확률과 비교한다.
- 실제 베팅 실행/계정 연동은 MVP 범위에서 제외한다.

2.3 비목표(Non-goals)

- 초기부터 모든 리그/종목 지원
- 실시간 라이브 베팅 의사결정 엔진

- "AI 단독 예측" 서비스
- 사용자 베팅 계정의 자동 주문/자동 배팅
- 규정에 위배될 수 있는 사이트의 무단 크롤링

3. 예측 엔진 설계

3.1 확률 계산 계층

레이어	역할	MVP
시장 기준선	현재 배당을 de-vig 해 시장 공정확률 계산	필수
Poisson / Dixon-Coles	예상 득점과 스코어 분포 -> 1X2/오버언더 확률	필수
Elo	장기 팀 강도와 상대 수준 반영	필수
ML (LightGBM)	비선형 변수 결합, baseline 잔차 보정	2 차
Calibration	모델의 60% 예측이 실제로 약 60%가 되도록 보정	필수
LLM 정보 레이어	뉴스/부상/인터뷰를 구조화 변수로 변환	2 차

3.2 주요 Feature

- 최근 5/10 경기 xG, 실제 득실, 슈팅/유효슈팅, 홈/원정 분리 성적
- 상대 전력 보정 Elo, 경기 간 휴식일, 유럽대항전/컵 일정 혼잡도
- 주전 선수 부상/징계, 예상 출전시간, 포지션별 결장 영향
- 예상 라인업과 실제 라인업 변화, 감독/전술 변경
- 개장 배당, 현재 배당, closing line, 배당 변화 속도
- 날씨/원정거리 등은 유의미성 검증 후 보조 변수로만 사용

3.3 핵심 수식/개념

시장 공정확률 decimal odds 의 1/odds 값을 구한 뒤 합이 1 이 되도록 정규화해 overround(마진)를 제거한다.

Edge 모델 확률 - 시장 공정확률. 단순 "적중 확률이 높은 경기"보다 시장과의 차이가 중요한 지표다.

Expected Value EV = (모델 확률 x decimal odds) - 1. EV 가 양수여도 표본 오차와 모델 오류가 있으므로 임계값/신뢰도 게이트를 둔다.

3.4 2 경기 조합 처리

MVP 에서는 서로 다른 경기의 선택지만 조합하고, 조합 확률은 독립 근사로 계산한다. 동일 경기 내 상관된 시장 또는 동일 팀과 강하게 연관된 조합은 제외한다. 향후에는 과거 동시 경기 데이터와 시장 상관 구조를 활용해 보정한다.

4. AI 계층 설계

4.1 AI 가 해야 하는 일

- 뉴스 기사에서 부상/복귀/징계/로테이션 관련 사실 추출
- 감독 인터뷰에서 출전 가능성, 로테이션 위험, 전술 변화 신호 추출
- 여러 출처를 요약하고 모순되는 정보는 불확실성으로 표시
- 모델 결과를 사용자에게 읽기 쉬운 한국어 설명으로 변환
- 출처 URL, 수집 시각, 추출된 사실, confidence 를 함께 저장

4.2 AI 가 하면 안 되는 일

- "느낌상" 승/무/패 확률을 직접 결정
- 출처 없는 부상/선발 정보를 사실처럼 생성
- 통계 모델의 확률을 근거 없이 크게 덮어쓰기

권장 구조 LLM -〉 Structured Event(JSON) -〉 Feature Builder -〉 Statistical/ML Model -〉 Probability -〉 Explanation. AI 는 입력 구조화와 설명을 담당하고 확률 엔진과 분리한다.

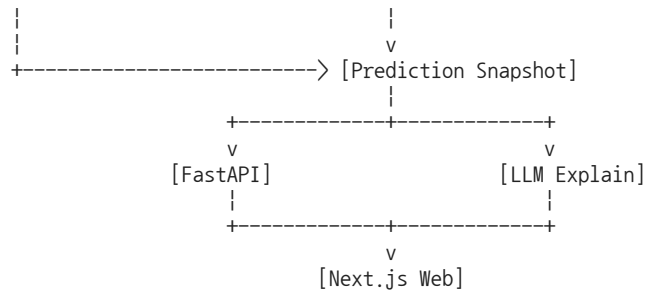
5. 기술 아키텍처

5.1 권장 스택

영역	기술	선정 이유
Web	Next.js + TypeScript	SEO, SSR/SSG, 경기별 랜딩 페이지, 광고/구독 UX 확장
API	FastAPI + Python	데이터/통계/ML 생태계와 동일 언어, 빠른 API 개발
DB	PostgreSQL	경기-팀-선수-배당-예측의 관계형 데이터와 분석 쿼리
Cache/Queue	Redis + Celery(또는 Dramatiq)	수집/재계산 작업 비동기 처리, API 호출 비용 절감
ML	pandas + scikit-learn + LightGBM + statsmodels	baseline 부터 ML 까지 검증 가능한 Python 생태계
Object Storage	S3 호환 스토리지	원본 응답, 모델 아티팩트, 리포트 보관
Analytics	PostHog 또는 GA 계열	검색 유입/경기 선택/전환 퍼널 분석
Monitoring	Sentry + structured logs	프론트/백엔드 오류 추적
CI/CD	GitHub Actions	테스트, lint, migration, 배포 자동화

5.2 논리 구조

```
[Data Providers]
      ¦
      v
[Ingestion Worker] -> [Raw Snapshots / S3]
      ¦
      v
[PostgreSQL] -> [Feature Builder] -> [Prediction Engine]
```



5.3 권장 저장소 구조

```
repo/
  apps/
    web/              # Next.js
    api/              # FastAPI
  workers/
    ingestion/        # fixtures, odds, lineup, injury jobs
  ml/
    features/
    models/
    evaluation/
  packages/
    contracts/        # API schema / shared enums
  infra/
    docker/
    migrations/
  docs/
  tests/
```

6. 데이터 설계

6.1 핵심 테이블

테이블	핵심 필드/목적
teams	provider_id, name, league, season
players	provider_id, team_id, position
fixtures	kickoff_at_utc, home_team_id, away_team_id, status, score
odds_snapshots	fixture_id, bookmaker, market, selection, odds, captured_at
team_match_stats	xG, shots, possession, rest_days 등 시점별 경기 통계
injury_events	player_id, status, source, observed_at, confidence
lineup_snapshots	fixture_id, expected/confirmed, captured_at
predictions	fixture_id, model_version, probabilities, odds_at_prediction, created_at
model_versions	git_sha, feature_set, training_window, metrics, artifact_uri
llm_events	source_url, extracted_json, model, prompt_version, confidence

6.2 데이터 원칙

- 모든 시간은 DB 에 UTC 로 저장하고 UI 에서 사용자 시간대로 변환한다.
- provider raw payload 는 가능한 범위에서 보존해 재처리 가능하게 한다.
- 배당/라인업/부상은 snapshot append 방식으로 저장하고 덮어쓰지 않는다.
- 상용화 전 데이터 제공사의 재배포/표시/상업적 사용 라이선스를 반드시 확인한다.

7. MVP 범위와 로드맵

Phase 0 - 개발 기반 (1~2 일)

- Monorepo, Docker Compose, Postgres, 환경변수, lint/test, CI 기본 구성
- Provider interface 와 Mock provider 구현
- DB migration 과 seed 데이터

Phase 1 - 확률 MVP (1~2 주)

- EPL 일정/결과/기본 배당 수집
- Poisson/Dixon-Coles + Elo baseline
- de-vig, Edge, EV 계산
- 경기 목록 + 경기 상세 페이지 + 2 경기 선택/조합 계산
- Prediction snapshot 과 model_version 저장

Phase 2 - 모델 고도화 (2~4 주)

- xG/라인업/부상/휴식일 feature 추가
- LightGBM + calibration + walk-forward validation
- Brier/Log loss/Calibration/CLV 대시보드

Phase 3 - AI/상용화 준비

- 뉴스/인터뷰 수집 및 structured extraction
- AI explanation + 출처 표시
- SEO 페이지, sitemap, structured data, analytics, 광고 슬롯
- 로그인/즐겨찾기/알림/프리미엄 기능은 트래픽 확인 후 추가

8. 검증과 성능 평가

8.1 모델 지표

지표	왜 중요한가
Brier Score	확률 예측의 오차를 직접 측정
Log Loss	과도하게 자신 있는 틀린 예측을 강하게 패널티
Calibration	예측 60%가 실제 약 60%로 실현되는지 확인
Closing Line Value	시장 마감 가격보다 더 좋은 정보를 지속적으로 포착하는지 확인
ROI	실전 성과 참고용. 단독 최적화하면 과적합 위험이 큼
Max Drawdown	운영/사용자 경험 측면의 변동성 확인

8.2 백테스트 규칙

- 랜덤 split 금지. 과거 -> 미래 순서로 walk-forward 검증한다.
- 그 당시 알 수 없었던 라인업/결과/closing odds 를 과거 예측 feature 에 넣지 않는다.
- 예측 생성 시각의 배당 snapshot 을 고정해 look-ahead bias 를 방지한다.
- paper prediction 300~500 경기 이상 누적 전에는 "모델 우위"를 단정하지 않는다.

9. 상용화, SEO, 광고

9.1 수익화 우선순위

6. 무료 스포츠 분석 콘텐츠 + 검색 유입 + 광고
7. 프리미엄: 고급 모델 지표, 라인업 업데이트, 배당 변화/알림, 히스토리
8. API/B2B: 확률/feature/리포트 제공
9. 베팅사 제휴/affiliate 는 법률 및 광고정책 검토 후 별도 판단

9.2 SEO 구조

- /matches/{home}-vs-{away}-{date} : 경기 상세
- /teams/{team} : 팀 프로필/최근 폼
- /predictions/epl/{date} : 날짜별 분석 허브
- /model/performance : 과거 예측 성능/방법론 공개
- SportsEvent 구조화 데이터, canonical URL, sitemap, 서버 렌더링/정적 생성을 활용

9.3 광고와 정책 리스크

> **주의** 스포츠 분석 콘텐츠와 직접적인 도박 권유/affiliate 콘텐츠는 광고 네트워크와 지역 규제에서 취급이 달라질 수 있다. 광고 도입 전에 서비스 문구, 연령 제한, 지역별 규정, 광고 플랫폼 정책을 검토한다.

- 서비스 문구는 "데이터 분석/확률" 중심으로 유지한다.
- 확정 수익, 적중 보장, 손실 복구 표현을 금지한다.
- 실제 베팅 링크/제휴를 넣는 경우 별도 법률/플랫폼 정책 검토를 거친다.

10. 보안/운영 기준

- API key 는 .env/secret manager 에만 저장하고 저장소 커밋 금지
- 외부 API 호출에는 timeout, retry, rate limit, circuit breaker 전략 적용
- 수집 job 은 idempotent 하게 설계하고 중복 snapshot 방지 키를 둔다.
- DB migration 은 Alembic 으로 관리하고 production 에서 자동 destructive migration 금지
- 사용자 정보는 최소 수집. MVP 에는 베팅 계정/결제수단/민감정보 연동 없음
- 예측 결과 페이지에 데이터 최신 시각과 누락/지연 상태를 명확히 표시

11. Claude Code 개발 요청 프롬프트 (v1)

아래 프롬프트는 새 저장소에서 Claude Code 에 그대로 전달하는 것을 기준으로 작성했다. 한 번에 상용 서비스 전체를 만들지 말고 Phase 0 + Phase 1 의 작동하는 MVP 까지를 목표로 한다.

```
당신은 시니어 풀스택 엔지니어이자 ML 엔지니어다. 아래 프로젝트를 production-minded MVP 로 설계하고 구현하라.
```

```
<project>
이름: EPL Edge Lab (working title)
목적: EPL 경기의 통계 데이터와 시장 배당을 이용해 1X2 확률, 시장 공정확률(de-vig), Edge, Expected Value를 계산하고, 사용자가 2~3개 경기를 선택해 분석할 수 있는 웹 서비스.
포지셔닝: 도박 실행 서비스가 아니라 스포츠 데이터/확률 분석 플랫폼.
핵심 원칙: LLM이 승률을 직접 찍지 않는다. 통계/ML 모델이 확률을 계산하고 LLM은 뉴스/부상/인터뷰 구조화와 설명을 담당한다.
</project>

<stack>
- Web: Next.js + TypeScript
- API: FastAPI + Python
- DB: PostgreSQL
- ORM/Migrations: SQLAlchemy + Alembic
- Queue/Cache: Redis + Celery (Phase 1에서 필요 최소 수준)
- ML baseline: pandas, numpy, scipy/statsmodels, scikit-learn
- Future ML: LightGBM
- Local dev: Docker Compose
- CI: GitHub Actions
- Tests: pytest + Vitest/Playwright as appropriate
</stack>

<architecture_constraints>
1. provider-specific 코드를 core domain과 분리한다. FootballDataProvider interface와 MockProvider를 먼저 만든다.
2. 실제 유료 API key가 없어도 전체 MVP가 mock/seed data로 실행돼야 한다.
3. 모든 시간은 DB에서 UTC로 저장한다.
4. odds, lineup, injury, prediction은 snapshot append 방식으로 저장하며 과거 값을 덮어쓰지 않는다.
5. predictions에는 반드시 fixture_id, model_version, created_at, input_data_as_of, probability_home/draw/away, market_odds_snapshot_id를 저장한다.
6. 모델 버전은 git sha 또는 명시적 semantic version과 feature set 정보를 기록한다.
7. secret은 코드/로그에 노출하지 않는다.
8. 데이터 수집 job은 재실행 가능한 idempotent 작업이어야 한다.
9. 실제 베팅 실행, 베팅 계정 연동, 자동 주문은 구현하지 않는다.
10. UI에서 확률을 "보장"으로 표현하지 않는다.
</architecture_constraints>

<mvp_phase_0>
- monorepo 구조 생성: apps/web, apps/api, workers/ingestion, ml, packages/contracts, infra, docs, tests
- Docker Compose로 Postgres + Redis + API + Web 로컬 실행
- 환경변수 템플릿(.env.example)
- 기본 CI, lint, format, test 명령
- README에 로컬 실행법
</mvp_phase_0>

<mvp_phase_1>
- DB 모델: teams, fixtures, odds_snapshots, predictions, model_versions
- MockProvider와 seed EPL fixtures/odds
- Poisson 기반 expected goals -> score matrix -> 1X2 확률
- 간단한 Elo rating 모듈
- 시장 odds에서 overround 제거(de-vig)
- Edge = model_probability - fair_market_probability
- EV = model_probability * decimal_odds - 1
- 최소한의 calibration interface를 만들되 초기에는 identity calibration 허용
- API:
  GET /health
  GET /fixtures?date=
  GET /fixtures/{id}
  GET /fixtures/{id}/prediction
  POST /combinations/analyze  # 서로 다른 경기의 2~3 selection
- 조합 확률은 MVP에서 independence approximation을 사용하고 응답에 approximation임을 명시
- Web:
  1) 오늘/이번 주 EPL 경기 목록
  2) 경기 상세: 1X2 모델확률, 시장 공정확률, Edge, EV, 데이터 as-of 시간
  3) 2~3경기 선택 후 조합 확률/배당/EV 계산 화면
  4) 모델 방법론/면책 페이지
- Prediction 생성 시 DB snapshot 저장
</mvp_phase_1>

<quality_gates>
- docker compose up 후 별도 수동 설정 없이 서비스가 떠야 한다.
- seed 명령 한 번으로 샘플 데이터가 생성돼야 한다.
- API unit tests와 model calculation tests를 작성한다.
- 확률 합이 1인지, de-vig 결과 합이 1인지 테스트한다.
- 동일 prediction job 재실행 시 중복/오염이 생기지 않도록 테스트한다.
```

```
- 프론트는 모바일에서도 읽을 수 있어야 한다.
- 임시 TODO 라도 "왜" 남는지 이슈/README 에 기록한다.
</quality_gates>
<working_method>
먼저 코드를 쓰지 말고:
A. 요구사항을 15 줄 이내로 요약
B. 예상 리스크/모호한 점을 최대 10 개 제시
C. repo tree 와 데이터 흐름을 제안
D. 구현 순서를 Phase 0 -> Phase 1 의 작은 체크포인트로 나눈다.
그 다음 구현을 시작한다.
각 체크포인트가 끝날 때마다:
- 변경 파일
- 실행/테스트 명령
- 테스트 결과
- 남은 리스크
- 다음 체크포인트
을 짧게 보고하라.
의사결정이 필요한 경우, 막연하게 질문하지 말고 2~3 개 옵션과 추천안을 함께 제시하라.
과도한 microservice/Kubernetes 도입은 금지한다. 단순하게 시작하되 코드 경계는 추후 분리 가능하게 유지한다.
```

12. 어떤 모델에 맡길까?

모델은 역할별로 나누는 것이 효율적이다. 아래 권장은 2026-10 기준이며, 모델 라인업은 자주 바뀌므로 실제 사용 시 최신 문서를 다시 확인한다.

용도	권장	이유
일상 구현/버그 수정/프론트 완성	Claude Sonnet 5.5	속도/비용 대비 코딩과 에이전트 작업이 강해 기본 개발 모델로 적합
초기 아키텍처/어려운 모델 설계/대형 리팩터링	Claude Opus 5.5	복잡한 판단, 장시간 작업, 대규모 코드 변경에 더 적합
독립적인 2 차 구현/리뷰	OpenAI Codex (GPT-5.3-Codex 계열)	repo 단위 장기 코딩 에이전트로 교차 검증/리뷰에 유용

추천 워크플로 Opus 로 설계/리뷰 -> Sonnet 으로 대부분의 구현 -> Codex 로 테스트/리팩터링/독립 리뷰. 예산을 줄이려면 Sonnet 하나로 시작해도 충분하다.

13. 바로 다음 액션

1. GitHub 에 새 저장소를 만든다. 이름은 가칭으로 시작해도 된다.
2. 이 문서와 개발 프롬프트를 저장소 docs/에 넣는다.
3. Claude Code 에서 위 Prompt v1 을 실행하고, 먼저 계획/리스크/repo tree 만 받는다.
4. 계획을 검토한 뒤 Phase 0 만 구현시킨다. docker compose + tests 가 통과하면 Phase 1 로 간다.
5. 실데이터 API 는 Provider interface 가 완성된 뒤 하나만 선택해 연결한다. 상용화 전에 라이선스를 검토한다.
6. 실제 베팅이 아니라 paper prediction 으로 예측 이력을 누적한다. 최소 수백 경기 이후 모델 우위를 평가한다.